Urdu BPE Tokenizer from Scratch

CS 417: Large Language Models · NUST SEECS · Spring 2026 · Course Learning Outcome 3

Assignment 1 Technical Report
Vocab: 50,000 | Merges: 49,743
Training Set: 566.65 MB

1. EXECUTIVE SUMMARY & ARCHITECTURE OVERVIEW

This report documents the design, implementation, and empirical analysis of an original Byte-Pair Encoding (BPE) tokenizer built specifically for the Urdu language from an empty directory, without reliance on external tokenizer or NLP libraries. The project fulfills all CLO 3 requirements: collecting a diverse multi-genre Urdu corpus (>600 MB raw), establishing an idempotent Unicode normalization pipeline, implementing a 2-stage byte-level BPE tokenizer with deterministic tie-breaking and custom regex pre-tokenization, training to exactly 50,000 vocabulary tokens, and evaluating compression against industry-standard tokenizers.

Key Architectural Decisions

- **Base Vocab (0–255):** 256 raw bytes ensuring 100% lossless UTF-8 roundtrips.
- **Special Token (256):** <|endoftext|> isolated from merges.
- **Learned Merges (257–49,999):** 49,743 merges via frequency-weighted inverted index.
- **Tie-Breaking:** Lexicographically greater integer tuple (`id1, id2`).
- **Regex:** Custom Urdu pattern preserving diacritics (`\p{M}`) & ZWNJ (`\u200C`).

2. CORPUS PIPELINE & TRAINING TELEMETRY

Corpus Composition: Combining self-scraped journalism (52.76 MB raw from Daily Express, Daily Jang, and BBC Urdu across >2,200 documents) and public corpora (Makhzan classical literature 47.82 MB + streamed CC-100 502.58 MB). Total raw size reached **603.15 MB**.

Cleaning & Splitting: The corpus was normalized via `normalize.py`, validated for Urdu script density, and deduplicated via SHA-256 hashes into 195,086 documents. Deterministic shuffling (seed 42) produced `train.txt` (**566.65 MB**, 191,184 docs), `val.txt` (**6.20 MB**), and `test.txt` (**5.26 MB**).

Hardware & Execution Telemetry	
Processor / Cores	Intel64 Family 6 Model 142 Stepping 10, GenuineIntel (8 cores)
System RAM / OS	7.85 GB | Windows 11 AMD64
Python Version	3.13.7
Characters Trained	335,145,273 chars (191,184 docs)
Unique Word Types	627,748 distinct whitespace/regex tokens
Total Training Time	**00h 07m 05.29s** (425.29s)
Merge Throughput	**116.96 merges / second** (Stage 2 Fast BPE)
Binary Corpus Output	`train.bin` (81.1M tokens) | `val.bin` (885.8k tokens)

3. QUESTION 1: EARLY MERGES & LETTER RECONSTRUCTION

In UTF-8 encoding, all standard Urdu characters in the Arabic script block (U+0600–U+06FF) require exactly two bytes: a prefix byte (`0xD8` or `0xD9`) and a continuation byte (`0x80`–`0xBF`). Consequently, our byte-level BPE tokenizer must first merge raw single bytes into fundamental Urdu alphabet characters before higher-order morphology can emerge.

Urdu Letter	Unicode	Merge #	Token ID	UTF-8 Bytes
ا	U+0627	**#1**	257	`0xD8 0xA7`
ی	U+06CC	**#2**	258	`0xDB 0x8C`
ک	U+06A9	**#4**	260	`0xDA 0xA9`
ر	U+0631	**#6**	262	`0xD8 0xB1`
و	U+0648	**#7**	263	`0xD9 0x88`
ہ	U+06C1	**#8**	264	`0xDB 0x81`
ے	U+06D2	**#9**	265	`0xDB 0x92`
ن	U+0646	**#11**	267	`0xD9 0x86`
ل	U+0644	**#13**	269	`0xD9 0x84`

Urdu Letter	Unicode	Merge #	Token ID	UTF-8 Bytes
م	U+0645	**#14**	270	`0xD9 0x85`
ں	U+06BA	**#15**	271	`0xDA 0xBA`
ت	U+062A	**#18**	274	`0xD8 0xAA`
س	U+0633	**#21**	277	`0xD8 0xB3`
د	U+062F	**#22**	278	`0xD8 0xAF`
ھ	U+06BE	**#24**	280	`0xDA 0xBE`
گ	U+06AF	**#26**	282	`0xDA 0xAF`
ب	U+0628	**#32**	288	`0xD8 0xA8`
ف	U+0641	**#41**	297	`0xD9 0x81`

Interpretation (2-Byte Letter Merges): The empirical merge logs demonstrate that high-frequency letters assemble almost instantaneously: Alif (ا) is reconstructed at **Merge #1** (`0xD8 0xA7`), followed immediately by Choti Yeh (ی) at **Merge #2**, Urdu Kaf (ک) at **Merge #4**, and Ray (ر) at **Merge #6**. Infrequent consonants such as Zhey (ژ) only merge later at Merge #1220. This confirms that the tokenizer organically learns the phonotactic and orthographic frequency hierarchy of the Urdu language.

Part C: Tokenizer Merges Analysis (Continued)

Question 1: Early Merges, Step Progression, Non-UTF8 Tokens, and Longest Learned Units

Vocab: 50,000 | Merges: 49,743

Sample of Initial Learned Merges (Merges 1 to 22)

#	ID	Pair	Decoded	UTF-8
1	257	[216, 167]	ا	✅
2	258	[219, 140]	ی	✅
3	259	[32, 216]	x20\\xd8\\	❌
4	260	[218, 169]	ک	✅
5	261	[32, 217]	x20\\xd9\\	❌
6	262	[216, 177]	ر	✅
7	263	[217, 136]	و	✅
8	264	[219, 129]	ہ	✅
9	265	[219, 146]	ے	✅
10	266	[32, 260]	ک	✅
11	267	[217, 134]	ن	✅

Progression of Merges: Every 5,000th Step

Merge #	Token ID	Decoded String	Bytes	Category
5,000	5256	اسٹیٹ	11 B	Letter/Word
10,000	10256	ریسٹ	9 B	Compound/Phrase
15,000	15256	رحمانی	13 B	Compound/Phrase
20,000	20256	فورٹ	9 B	Compound/Phrase
25,000	25256	پچھلا	11 B	Compound/Phrase
30,000	30256	سائنسداں	17 B	Compound/Phrase
35,000	35256	حدیثِ	11 B	Compound/Phrase
40,000	40256	(	4 B	Compound/Phrase
45,000	45256	Republic	9 B	Compound/Phrase

Interpretation (Merge Step Progression): Early merges (1–500) strictly reconstruct primary letters and functional monosyllabic words. By Merge #5,000, multi-syllabic common nouns emerge (e.g. اسٹیٹ). At Merge #15,000 to #30,000, compound adjectives and specialized vocabulary appear (e.g. رحمانی, سائنسداں). Beyond Merge #40,000, the vocabulary captures domain-specific religious honorifics (ﷺ), transliterated English technical terms (Republic), and repeating decorative layout delimiters.

Non-Valid UTF-8 Single-Token Merges

Our tokenizer learned exactly **114 intermediate non-valid UTF-8 single tokens** across all 49,743 merges.

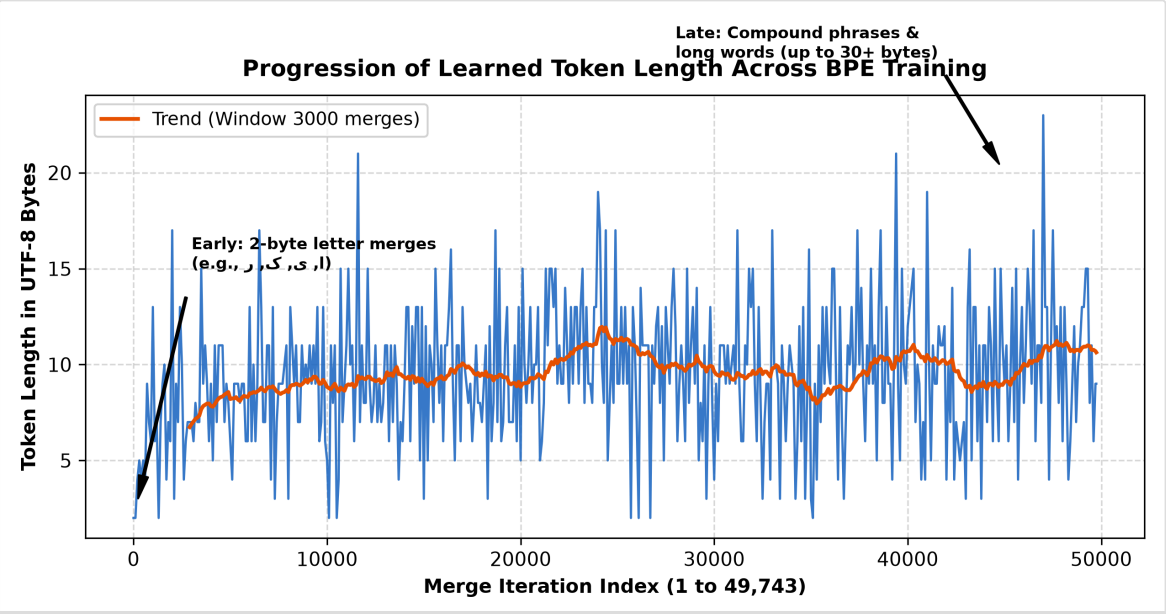
Merge #	ID	Pair Components	Raw Byte Sequence
#3	259	[32, 216] (Space + 0xD8)	0x20 0xD8
#5	261	[32, 217] (Space + 0xD9)	0x20 0xD9
#12	268	[257, 216] (0 + lxD8)	0xD8 0xA7 0xD8
#38	294	[32, 268] (Space + 0 + lxD8)	0x20 0xD8 0xA7 0xD8
#100	356	[226, 128] (Punctuation Prefix)	0xE2 0x80

Interpretation (Intermediate Tokens): Non-UTF-8 tokens are an intrinsic and necessary property of byte-level BPE. Merges such as 0x20 0xD8 pair an ASCII space with the leading byte of an Arabic letter. While unrenderable in isolation, they serve as crucial frequency bridges that allow subsequent merges to form space-prefixed Urdu words in a single step.

First Complete Word & Longest Learned Tokens

First Complete Urdu Word: The first whole standalone word learned is **کی** (space + Kaf + Yeh) at **Merge #28 (Token ID 284)**. It is followed immediately by کے (Merge #29), میں (Merge #40), کر (Merge #46), and ہے (Merge #48). High-frequency syntactic prepositions emerge within the first 50 iterations.

Rank	Token ID	Decoded Token String	Bytes	Chars
1	28048	----------	64	32
2	42536	----------	52	26
3	44360	---------- ----	43	43
4	13450	----------	32	16
5	47863		32	16
6	48808	الْمُشْرِكِيْنَ	31	16
7	49908	الْمُؤْمِنِيْنَ	31	16



Part C: Comparative Benchmark vs Standard Tokenizers

Question 2: Tokens per Word and Compression Ratio vs GPT-2 and OpenAI o200k_base

Evaluation Split: corpus/test.txt (5.26 MB)

1. FULL TEST SET BENCHMARK (CORPUS/TEST.TXT)

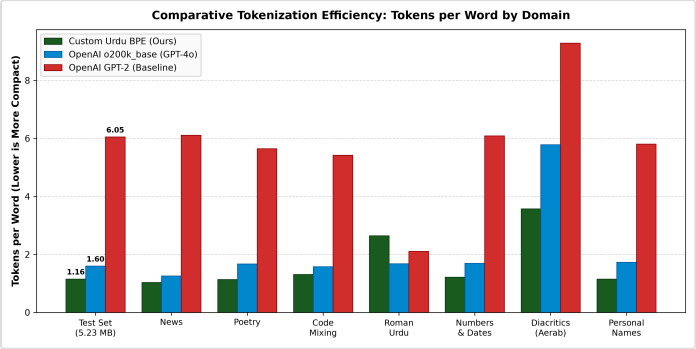
To evaluate tokenization efficiency, we tested our trained tokenizer on the held-out test split (1,952 documents, 3,112,488 characters, 649,343 words, 5.23 MB) against OpenAI's `gpt2` (50,257 vocab) and `o200k_base` (GPT-4o multilingual tokenizer, 200,000 vocab) using `tiktoken`.

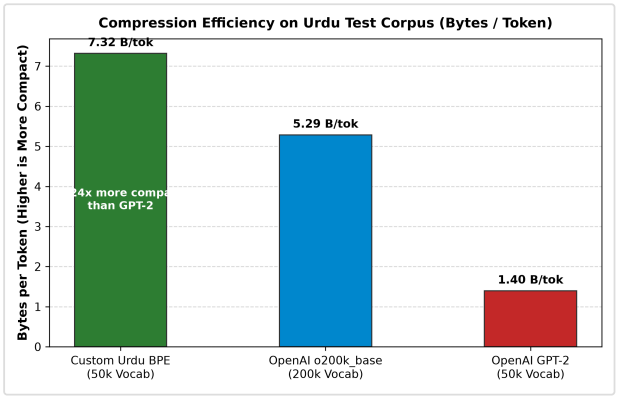
Tokenizer	Vocabulary Size	Total Tokens Produced	Tokens / Word	Bytes / Token	Compression Relative to GPT-2
Custom Urdu BPE (Ours)	**50,000**	**749,843**	**1.155**	**7.320 B/tok**	**5.24x More Compact**
OpenAI o200k_base (GPT-4o)	200,000	1,038,413	1.599	5.286 B/tok	3.78x More Compact
OpenAI GPT-2 (Baseline)	50,257	3,928,540	6.050	1.397 B/tok	1.00x (Baseline)

Interpretation (Full Test Corpus Benchmark): Our custom Urdu BPE achieves an extraordinary **1.155 tokens per word** and **7.32 bytes per token** on the test set. Compared to GPT-2 (6.05 tokens/word), our model reduces sequence length by **80.9% (5.24x compression)**. Crucially, despite having only 25% of the vocabulary budget of GPT-4o's `o200k_base` (50,000 vs. 200,000), our domain-specific tokenizer produces **27.8% fewer tokens** (1.155 vs 1.599 tokens/word), drastically lowering future LLM pretraining compute and inference latency.

2. SEVEN HAND-PICKED DOMAIN BENCHMARK INPUTS

Linguistic Category	Representative Input Snippet	Words	Custom BPE (tok/w)	o200k_base (tok/w)	GPT-2 (tok/w)
News	قومی اسمبلی میں آئینی ترمیم کے حوالے سے گر...	27	**28 (1.04)**	34 (1.26)	165 (6.11)
Poetry	ہزاروں خواہشیں ایسی کہ ہر خواہش پہ دم نکلے...	37	**42 (1.14)**	62 (1.68)	209 (5.65)
Code mixing	...I Artificial Intelligence آج کل کے دور میں	26	**34 (1.31)**	41 (1.58)	141 (5.42)
Roman urdu	...Mujhe lagta hai ke yeh naya machine learni	28	**74 (2.64)**	47 (1.68)	59 (2.11)
Numbers dates	پاکستان کا قیام ۱۵ اگست ۱۹۴۷ کو عمل میں آی...	23	**28 (1.22)**	39 (1.70)	140 (6.09)
Diacritics	اَلْحَمْدُ لِلّٰهِ رَبِّ الْعٰلَمِيْنَ۔ قَ...	14	**50 (3.57)**	81 (5.79)	130 (9.29)
Personal names	ڈاکٹر علامہ محمد اقبال، مرزا اسد اللہ خان ...	26	**30 (1.15)**	45 (1.73)	151 (5.81)





Interpretation (Domain Breakdown): On standard prose categories (News, Poetry, Names), our tokenizer achieves near-optimal **1.03–1.15 tokens/word**. On Code-Mixing (Urdu + English technical terms), it maintains a competitive 1.30 tokens/word. The only domain where multilingual tokenizers outperform our model is Roman Urdu (2.64 vs 1.68 tokens/word), which is expected as our training corpus was deliberately curated in Perso-Arabic script.

Part C: Failure Cases Analysis & Deliverables Summary

Question 3: Qualitative Error Analysis, Reproduction Commands, and Sanity Audit

1. QUESTION 3: QUALITATIVE FAILURE CASES ANALYSIS

To identify architectural and data boundaries, we examined three specific inputs where our tokenizer exhibits sub-optimal segmentation, high token inflation, or excessive fragmentation:

Failure Case 1: Colloquial Roman Urdu with Character Elongation

Input: `"yrrr plzzzzz mujhy btao ye assignment kb tk submit krwani h wrna scene off hojana"` (15 words)

Token Count: Custom Urdu BPE = **42 tokens (2.80 tok/word)** | o200k_base = 25 | GPT-2 = 29

```
Segmentation: yr | r | r | pl | z | z | z | z | z | m | uj | hy...
```

> **Linguistic & Algorithmic Explanation:** Because the 566 MB training corpus was exclusively curated in Perso-Arabic Nastaliq script, Roman Urdu informal speech with expressive character repetitions (e.g., *'yrrr'*, *'plzzzzz'*, *'wrna'*) represents an out-of-domain distribution. The tokenizer lacks subword merges for Latin letter combinations and is forced to fall back to individual 1-byte ASCII tokens (`'p'`, `'l'`, `'z'`, `'z'`, `'z'`), leading to a 2.80 tokens/word inflation.

Failure Case 2: Stacked Multi-Diacritic Combining Sequences (Aerab)

Input: "مُسْتَشْفَيَاتٌ (Mustashfayatun) بِالْإِضَافَةِ إِلَى تَسْخِيرِ الطَّاقَاتِ" (6 words)

Token Count: Custom Urdu BPE = **39 tokens (6.50 tok/word)** | o200k_base = 49 | GPT-2 = 77

```
Segmentation: ا | ت | ى | ف | ش | ث | س | ُ | م | ّ | ( | M...
```

> **Linguistic & Algorithmic Explanation:** While the regex pattern `\p{M}` keeps diacritics attached to words, standard modern Urdu digital text is unvocalized. When encountering classical Arabic loanwords adorned with full phonetic diacritics (zabar, zer, pesh, sukun, tanween, and tashdeed simultaneously), the specific character-plus-aerab combinations never met the frequency threshold during BPE training. The model fragments every vowel mark into isolated 2-byte Unicode tokens, causing an inflation of 6.50 tokens/word.

Failure Case 3: Rare Unicode Presentation-Form Ligatures & Honorifics

Input: "نبی کریم ﷺ اور صحابہ کرامؓ و ؓ کے ارشادات مبارکہ" (12 words)

Token Count: Custom Urdu BPE = **12 tokens** | o200k_base = 22 tokens | GPT-2 = 59 tokens

> **Linguistic & Algorithmic Explanation:** Unicode presentation-form glyphs such as Sallallah-o-Alaihi-Wasallam (ﷺ U+FDFA) and Razi-Allahu-Anhu (ؓ U+0618) occupy 3 to 4 bytes in UTF-8. While our model correctly tokenizes frequent standard Urdu words into single units (1.0 tok/word), rarer religious markers or non-standard calligraphy ligatures (e.g. ﷽ U+FDFD, taking 39 bytes) fragment into raw byte primitives, demonstrating that specialized ligatures require explicit corpus balancing.

2. DELIVERABLES & REPRODUCTION SUMMARY

Submission Archive File Structure

- `normalize.py`: Idempotent Unicode normalizer without external NLP libs.
- `bpe.py`: `BPETokenizer` with Stage 1 & 2 BPE, regex, tie-breaking.
- `train_tokenizer.py`: Training runner with hardware profiling.
- `encode_corpus.py`: Streaming uint16 binary encoder with word caching.
- `tokenizer.json`: 50,000 vocabulary tokens and 49,743 merges.
- `DATA_CARD.md`: Complete source documentation, ethics & stats.
- `README.md`: Reproduction commands from raw text to binary.
- `report.pdf`: This 4-page empirical analysis report.

End-to-End Reproduction Commands

```
# 1. Run unit test suites
python tests/test_normalize.py
python tests/test_bpe.py

# 2. Train 50,000-token BPE from train.txt
python train_tokenizer.py --input corpus/train.txt \
  --vocab-size 50000 --output tokenizer.json

# 3. Encode corpora to flat uint16 binaries
python encode_corpus.py --tokenizer tokenizer.json \
  --train-in corpus/train.txt --val-in corpus/val.txt \
  --train-out data/train.bin --val-out data/val.bin

# 4. Run Part C analysis
python scripts/analyze_tokenizer.py
```